SCOPPHU · AI ENGINEERING COURSE 2026 · MÓDULO 02 · CONTEÚDO EXTRA

Apache Airflow

O que é, como está feito por dentro, e como se escreve

Material de apoio à Aula 02. Não é preciso em sala — serve para quem quer perceber a ferramenta, e como referência de sintaxe quando estiverem a escrever o vosso primeiro DAG.

Airflow **3.3** · é a versão do projeto, e a sintaxe mudou face ao Airflow 2.

O QUE É

O problema: um script não é uma pipeline

O QUE UM CRON FAZ

Corre um comando a uma hora.

E mais nada.

O QUE UM CRON NÃO FAZ

- não sabe que a tarefa B depende da A
- não tenta outra vez se falhar
- não avisa ninguém
- não guarda histórico
- não sabe reprocessar o passado
- não impede duas execuções em paralelo

Um **orquestrador** é um cron que sabe o que correu, o que falhou e o que falta.

O QUE É

O que o Airflow é — e o que não é

É

- um **agendador** de fluxos de trabalho
- escrito e definido em **Python**
- com **histórico**, logs e UI
- com **retries** e dependências
- capaz de **reprocessar o passado**

NÃO É

- uma ferramenta de **processamento** de dados
- um motor de **streaming** (é batch)
- um **ETL** — o ETL é o teu código
- uma base de dados
- um sítio para correr uma query de 4 horas num worker

Airflow orquestra. Quem transforma é o teu código.

O QUE É

Cinco palavras, e já se percebe tudo

DAG
O fluxo.
Um grafo de tarefas, sem ciclos. É um ficheiro Python.

TASK
Um passo.
Um nó do grafo.

OPERATOR
O tipo de passo.
Correr Python, um bash, uma query…

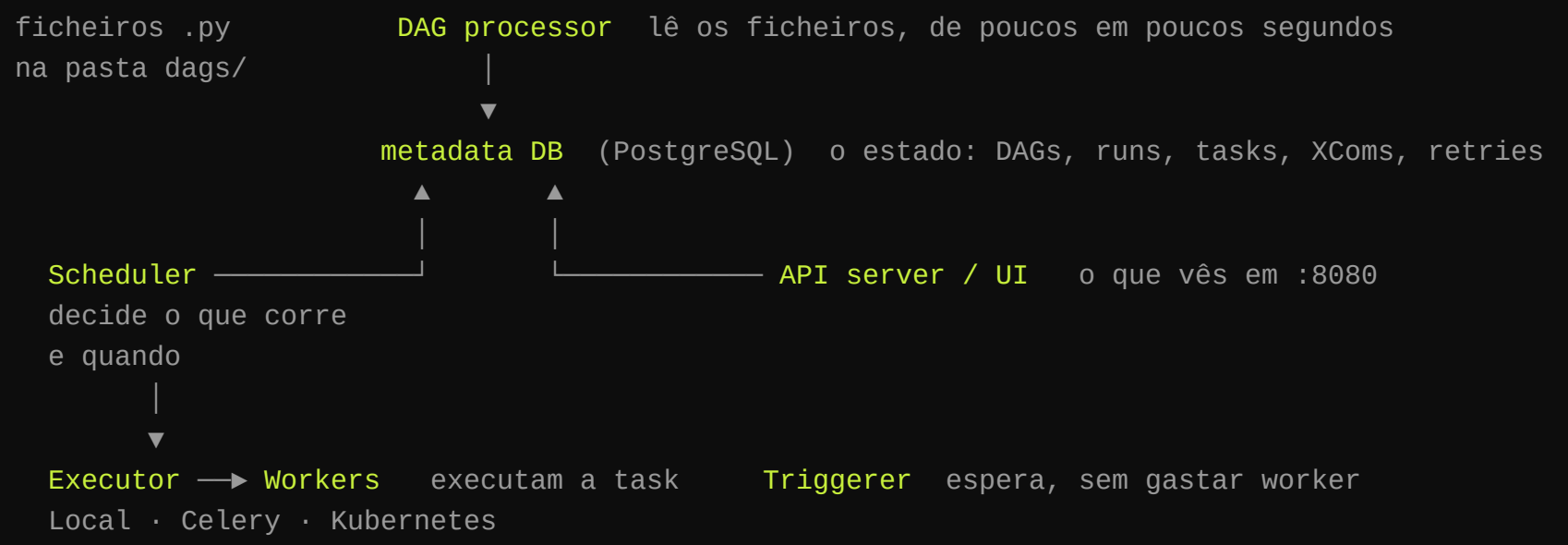
DAG RUN
Uma execução do fluxo.
Uma por período agendado.

DATA INTERVAL
A fatia de tempo que a run processa.
`[início, fim)`

```
DAG  tracking_pipeline
 │
 ├── task extract      (operator: PythonOperator)
 ├── task validate     ──▶  DAG run de 3 Out  ──▶  data interval [3 Out 00:00, 4 Out 00:00)
 └── task load
```

POR DENTRO

Os componentes



NESTA FORMAÇÃO

`airflow standalone` : tudo num container, `LocalExecutor` , sem login.

EM PRODUÇÃO

Componentes separados, executor distribuído, base de metadados gerida com backups, autenticação.

A PEÇA CENTRAL

A **metadata DB**. É ela que sabe tudo. Perdê-la é perder o histórico — e é por isso que leva backup.

SINTAXE

Um DAG é um ficheiro Python

```python
from datetime import datetime, timedelta
from airflow.sdk import DAG
from airflow.providers.standard.operators.python import PythonOperator

with DAG(
    dag_id="exemplo",                       # unico; e o nome na UI
    start_date=datetime(2026, 10, 1),       # a partir de quando faz sentido correr
    schedule="@daily",                      # de quanto em quanto tempo
    catchup=False,                          # nao recuperar o passado ao ativar
) as dag:
    a = PythonOperator(task_id="a", python_callable=funcao_a)
    b = PythonOperator(task_id="b", python_callable=funcao_b)
    a >> b                                  # b so corre depois de a ter sucesso
```

O ficheiro vive na pasta `dags/`. O DAG processor lê-o de poucos em poucos segundos — **o ficheiro é executado**, não é lido como texto.

SINTAXE

O mesmo DAG, com a TaskFlow API

```python
from airflow.sdk import dag, task

@dag(start_date=datetime(2026, 10, 1), schedule="@daily", catchup=False)
def exemplo():
    @task
    def extrair() -> dict:
        return {"linhas": 172}           # o retorno vai para o XCom, sozinho

    @task
    def carregar(dados: dict):
        print(dados["linhas"])           # recebe-o como argumento

    carregar(extrair())                  # a dependencia e a chamada da funcao

exemplo()
```

GANHA-SE
XCom implícito, menos cerimónia.

PERDE-SE
A fronteira entre orquestração e lógica fica menos óbvia.

NO PROJETO
Estilo clássico: ver o `>>` ensina melhor.

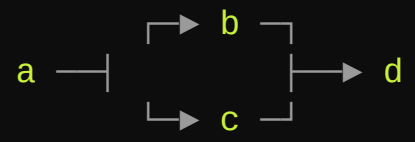
SINTAXE

Dependências

```
a >> b                      # b depois de a
a << b                      # a depois de b  (menos usado, confunde)

a >> [b, c] >> d            # b e c em paralelo, d espera pelos dois

from airflow.sdk import chain, cross_downstream
chain(a, b, c, d)           # a >> b >> c >> d
cross_downstream([a, b], [c, d])   # cada um dos primeiros liga a cada um dos segundos
```



Uma task só arranca quando **todas** as anteriores tiveram sucesso. Se uma falha, as seguintes ficam `upstream_failed` — nunca chegam a correr.

SINTAXE

Operators: o tipo de cada passo

Operator	Para quê	Vem de
PythonOperator	correr uma função Python	`providers.standard`
BashOperator	correr um comando	`providers.standard`
EmptyOperator	um nó que não faz nada (marcar início/fim)	`providers.standard`
SQLExecuteQueryOperator	correr SQL numa connection	`providers.common.sql`
DockerOperator · KubernetesPodOperator	correr um container	provider próprio
S3 · BigQuery · Snowflake · Slack · dbt…	centenas, um por sistema	provider próprio

No Airflow 3 quase tudo vive em **providers** — pacotes instalados à parte. Daí os imports longos: `from airflow.providers.standard.operators.python import PythonOperator`.

Na dúvida: PythonOperator a chamar o teu módulo.

SINTAXE

Quando corre: schedule

```
schedule="@daily"                  # presets: @hourly @daily @weekly @monthly
schedule="0 4 * * *"               # cron: todos os dias as 04:00
schedule=timedelta(hours=6)        # de 6 em 6 horas
schedule=None                      # so a mao
schedule=[meu_asset]               # quando outro DAG produzir este asset
```

ARMADILHA DO AIRFLOW 3

Um `schedule="@daily"` simples **já não cria data intervals**.

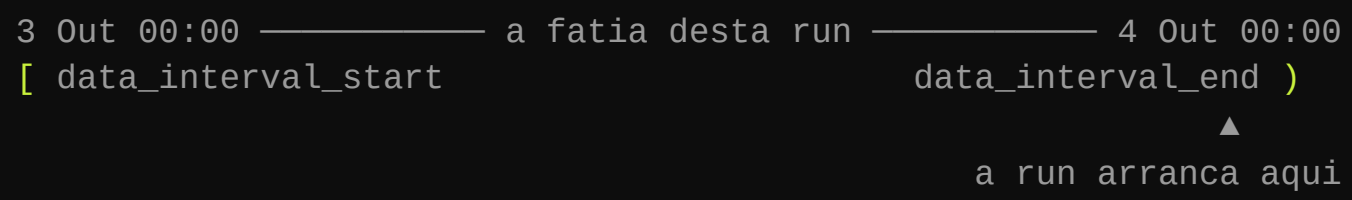
O QUE O PROJETO USA

```
from airflow.timetables.interval \
    import CronDataIntervalTimetable

schedule=CronDataIntervalTimetable(
    "@daily", timezone="UTC")
```

SINTAXE

Que fatia corre: data interval e templating




```
BashOperator(task_id="x", bash_command="echo {{ ds }}")      # 2026-10-03

def minha_funcao(data_interval_start=None, data_interval_end=None, **ctx):
    ...                       # o Airflow injeta o contexto nos argumentos
```

Variável	Valor	Variável	Valor
{{ ds }}	2026-10-03	{{ data_interval_start }}	datetime do início
{{ ds_nodash }}	20261003	{{ data_interval_end }}	datetime do fim
{{ params.x }}	parâmetro do DAG	{{ dag_run.conf }}	o que veio no trigger

SINTAXE

XCom: passar informação entre tasks

```
def a(ti):
    ti.xcom_push(key="caminho", value="/staging/20261003/clicks.json")
    return {"linhas": 172}                    # vai para a chave "return_value"

def b(ti):
    meta = ti.xcom_pull(task_ids="a")         # {"linhas": 172}
    caminho = ti.xcom_pull(task_ids="a", key="caminho")
```

O QUE LÁ PÔR

Caminhos, contagens, identificadores, pequenos dicionários. **Metadados.**

O QUE NÃO LÁ PÔR

DataFrames, listas de milhares de registos, ficheiros. O XCom **é a base de metadados** — enche-se depressa.

O padrão correto: os dados vão para um sítio partilhado (ficheiro, S3, uma tabela) e **pelo XCom viaja só o caminho**.

SINTAXE

Quando falha: retries e timeouts

```
default_args = {
    "retries": 2,                              # 1 tentativa + 2 = 3 no total
    "retry_delay": timedelta(minutes=5),
    "retry_exponential_backoff": True,         # 5, 10, 20 min...
    "max_retry_delay": timedelta(hours=1),
    "execution_timeout": timedelta(minutes=30), # mata a tentativa
}

with DAG(..., default_args=default_args) as dag:
    t = PythonOperator(task_id="t", python_callable=f, retries=0)   # anula o default
```

RETRIES
Para erros **transitórios**: 500, timeout, 429, deadlock.

RETRIES=0
Para erros **determinísticos**: schema errado, valor impossível, bug.

EXECUTION_TIMEOUT
Sem ele, uma chamada pendurada fica a correr para sempre e ocupa um slot.

SINTAXE

Trigger rules: quando é que uma task arranca

Regra	A task corre quando…	Para quê
all_success	todas as anteriores tiveram sucesso	**o default**
all_done	todas acabaram, com sucesso ou não	limpeza, notificação final
one_failed	pelo menos uma falhou	alerta
one_success	pelo menos uma teve sucesso	caminhos alternativos
none_failed_min_one_success	nenhuma falhou e pelo menos uma correu	depois de um *branch*
always	sempre	raro, e perigoso

```
notificar = PythonOperator(task_id="notificar", python_callable=avisar,
                           trigger_rule="all_done")
```

SINTAXE

Esperar por alguma coisa: sensors

```
from airflow.providers.standard.sensors.filesystem import FileSensor

espera = FileSensor(
    task_id="espera_ficheiro",
    filepath="/data/entrada/{{ ds }}.csv",
    poke_interval=60,            # verifica de minuto a minuto
    timeout=60 * 60 * 6,         # desiste ao fim de 6 horas
    mode="reschedule",           # LIBERTA o slot entre verificacoes
)
```

MODE="POKE" (DEFAULT)

Ocupa um worker durante toda a espera. Seis sensores a esperar seis horas bloqueiam seis slots.

MODE="RESCHEDULE" OU DEFERRABLE

Liberta o slot entre verificações. Os *deferrable* passam a espera para o **triggerer** e são o caminho moderno.

SINTAXE

Organizar: branching e task groups

```python
from airflow.sdk import task

@task.branch
def escolher(**ctx) -> str:
    if e_fim_do_mes(ctx["ds"]):
        return "fecho_mensal"
    return "fecho_diario"
```

Devolve o `task_id` do caminho a seguir. Os outros ficam `skipped`.

```python
from airflow.sdk import TaskGroup

with TaskGroup("carregar") as carregar:
    PythonOperator(task_id="dimensoes", ...)
    PythonOperator(task_id="factos", ...)

extrair >> carregar >> verificar
```

Agrupa tasks numa caixa que se abre e fecha na UI. É só organização visual — e vale muito num DAG com 40 tasks.

SINTAXE

Onde vivem os segredos: connections e variables

```
from airflow.sdk import Variable
from airflow.providers.postgres.hooks.postgres import PostgresHook

limite = Variable.get("limite_rejeicao", default_var=0.01)
hook   = PostgresHook(postgres_conn_id="analytics")   # host, user, pass vem da connection
df     = hook.get_pandas_df("SELECT * FROM fact_conversions LIMIT 10")
```

CONNECTION
Como falar com um sistema: host, porta, utilizador, password, extras. Identificada por um `conn_id`.

VARIABLE
Um valor de configuração com nome. Útil — mas ler uma no topo do ficheiro é uma query à base a cada leitura.

NUNCA
Credenciais no código ou no git. Em produção: um *secrets backend* (Vault, AWS Secrets Manager).

O nosso projeto não usa connections: lê o `DATABASE_URL` do ambiente. É mais simples de ensinar — e é também uma simplificação.

OPERAÇÃO

A CLI que vale a pena saber

```
airflow dags list                          # que DAGs existem
airflow dags list-import-errors            # porque e que o meu DAG nao aparece
airflow dags unpause tracking_pipeline     # ativar
airflow dags trigger tracking_pipeline --conf '{"date": "2026-10-03"}'

airflow tasks test tracking_pipeline extract 2026-10-03    # corre UMA task, sem gravar estado
airflow dags test tracking_pipeline 2026-10-03             # corre o DAG inteiro, local

airflow backfill create --dag-id tracking_pipeline \
    --from-date 2026-10-01 --to-date 2026-10-05

airflow db check                           # a base de metadados responde?
```

`tasks test` é a melhor ferramenta de depuração do Airflow: corre a task no terminal, com o contexto todo, sem tocar no histórico.

OPERAÇÃO

Os cinco erros de quem começa

1 · CÓDIGO NO TOPO

Uma chamada à API ou um `read_csv` ao nível do módulo corre **a cada leitura do ficheiro**, não a cada run.

2 · `DATETIME.NOW()`

Dentro de uma task, destrói o backfill: reprocessar ontem passa a processar hoje.

3 · DADOS NO XCOM

Um DataFrame no XCom é um DataFrame na base de metadados.

4 · `CATCHUP=TRUE` DISTRAÍDO

Um `start_date` de 2020 e o scheduler cria centenas de runs ao ativar o DAG.

5 · A TASK QUE FAZ TUDO

Uma só task com 400 linhas: não há retry parcial, não há paralelismo, e o log é ilegível.

E O BÓNUS

Lógica de negócio dentro do DAG. Se não se testa sem Airflow, está no sítio errado.

FECHO

Em três frases

Um DAG é um ficheiro Python que descreve passos e dependências.

Uma run processa uma fatia de tempo, não «agora».

O Airflow decide e regista.
Quem faz o trabalho é o teu código.

A seguir: abrir `dags/tracking_pipeline.py` e reconhecer cada uma destas peças. São 137 linhas e estão lá todas.